Robot Navigation Algorithm based on APF

The robot is assumed to be a point, with its position represented by $\vec{r} = \begin{bmatrix} x \\ y \end{bmatrix}$, since the robot is a wheeled robot that moves in 2D in the real world.

Input

The robot receives a point cloud at each iteration. This is because, at each iteration, a sensor is used to map the surroundings and a list of detected points' coordinates is received as input. These are points that must be avoided.

Additionally, the initial position and the goal position, along with the bounds of the space (X, Y) (all in Cartesian coordinates) must be provided, since this robot does not actively run any localization algorithms.

Output

At each iteration, the goal is to find a displacement vector $\vec{s}$ such that the new position of the robot is given by

$$\overrightarrow{r_{n+1}} = \overrightarrow{r_n} + \vec{s} \tag{1}$$

This is evaluated at each iteration. This algorithm can be run online, since new points added at each iteration will influence the next iteration.

Constructing the displacement vector $\vec{s}$

Based on intuition, it was decided that the robot's motion is controlled by three different objectives:

1. Movement towards the goal position i.e. attraction
2. Movement away from the walls i.e. repulsion
3. Movement away from the detected points i.e. repulsion from obstacles

Points 2. and 3. are necessary to ensure that the robot does not crash.

The resultant $\vec{s}$ is simply defined to be a weighted sum of these three forces.

$$\vec{s} = k \cdot \vec{g} + m \cdot \left(\overrightarrow{w_x} + \overrightarrow{w_y}\right) + n \cdot \sum \overrightarrow{b_i} \tag{2}$$

In this equation,

$k, m, n \in]0, 1[$ are constants such that $k + m + n = 1$

$\vec{g}, (\overrightarrow{w_x} + \overrightarrow{w_y}), \sum \overrightarrow{b_i}$ are normalized vectors which represent the motions described in points 1.-3. respectively. These are explained below.

These vectors need to be normalized to account for the varying degrees of magnitude. As described below, they are defined in terms of differences of position vectors. For example, $\vec{g}$ might extend across the entire field, while $\overrightarrow{b_i}$ might be negligible in magnitude in comparison, although significant for navigation.

Attraction towards the goal

Let the goal's position vector be $\vec{G}$. To embody motion directed towards the goal, a difference vector was chosen:

$$\vec{g} = \vec{G} - \overrightarrow{r_n}$$

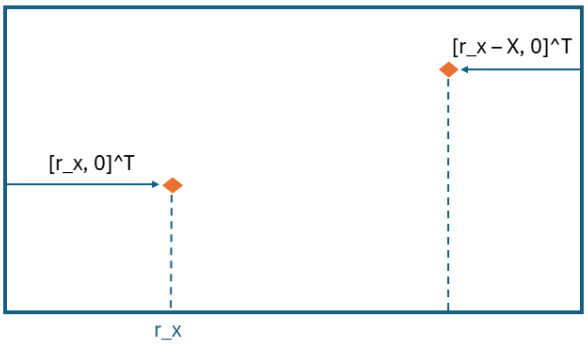
This is then normalized.

$$\vec{g} \leftarrow \frac{\vec{g}}{|\vec{g}|}$$

In a nutshell, $\vec{g}$ is a normalized vector pointing from the robot to the goal.

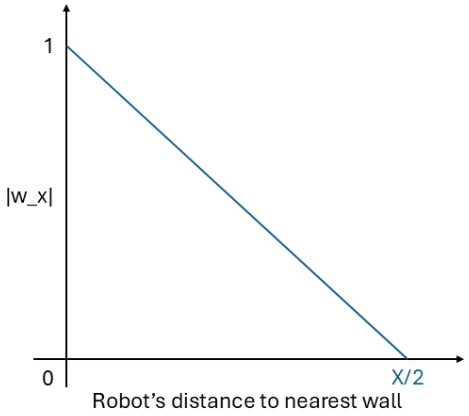
Repulsion away from the walls

Repulsion acting in ***x*-direction**: A vector $\overrightarrow{w_x}$ is evaluated that points from the nearest vertical wall to the robots. This is described below.



$$\overrightarrow{w_x} = \begin{cases} \begin{bmatrix} r_x - 0 \\ 0 \end{bmatrix}, & r_x < \dfrac{X}{2} \\ \begin{bmatrix} r_x - X \\ 0 \end{bmatrix}, & r_x \geq \dfrac{X}{2} \end{cases}$$

Now, this vector must be normalized. Since this denotes repulsion, it is expected that has the highest magnitude when the robot is closest to the wall. The following behaviour is required.



This can be accomplished by multiplying the original vector by the following normalizing factor.

$$\overrightarrow{w_x} \leftarrow \overrightarrow{w_x} \cdot \left(\frac{\frac{X}{2} - |\overrightarrow{w_x}|}{\frac{X}{2}} \right)$$

Repulsion acting in ***y*-direction**: This is defined in a similar manner, simply switching to y-coordinates.

$$\overrightarrow{w_y} = \begin{cases} \begin{bmatrix} r_y - 0 \\ 0 \end{bmatrix}, & r_y < \dfrac{Y}{2} \\ \begin{bmatrix} r_y - Y \\ 0 \end{bmatrix}, & r_y \geq \dfrac{Y}{2} \end{cases}$$

$$\overrightarrow{w_y} \leftarrow \overrightarrow{w_y} \cdot \left(\frac{\frac{Y}{2} - |\overrightarrow{w_y}|}{\frac{Y}{2}} \right)$$

Repulsion away from obstacle points

Initially, a zone of interest around the robot is defined with threshold radius R_{th}. Only obstacle points within this radius at the current position of the robot

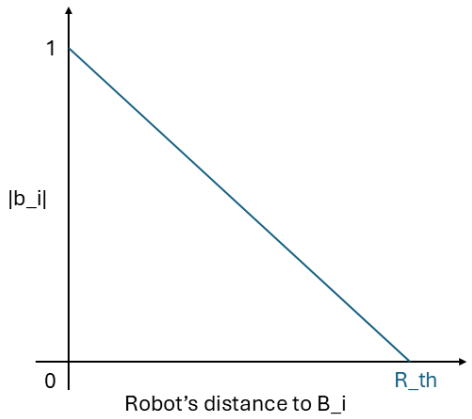
can influence its displacement in that iteration. This implies that in any given iteration, the maximum distance between an obstacle point and the robot is R_{th}.

Repulsion vectors from each of the obstacle points within the zone of interest are calculated. These vectors point from the point to the robot, hence pushing it away.

$$\overrightarrow{b_i} = \vec{r} - \overrightarrow{B_i}$$

Here $B_0, B_1, \cdots$ denote the coordinates of all the obstacle points in the zone of interest.

While normalizing, the behaviour expected is like that in the case of repulsion from walls.



$$\overrightarrow{b_i} \leftarrow \overrightarrow{b_i} \cdot \frac{R_{th} - |\overrightarrow{b_i}|}{R_{th}}$$

Compiling all the elements

By adding all these elements together, a final displacement vector was calculated.

$$\vec{s} = k \cdot \vec{g} + m \cdot \left(\overrightarrow{w_x} + \overrightarrow{w_y}\right) + n \cdot \sum \overrightarrow{b_i}$$

Finally, $\vec{s}$ is multiplied to bring it to a given length l. This ensures that the robot moves a comfortable fixed distance in every iteration, although this might cause overshooting in tight spaces.

$$\vec{s} \leftarrow \vec{s} \cdot \frac{l}{|\vec{s}|}$$

The actual values of all the constants here, like k, m, n, R_{th}, l should be defined after testing in the real-world. These are dependent on map size, sensor confidence and accuracy, motor control etc.